

Università di Bologna

Linguaggio C per Sistemi Operativi

Appunti dalle lezioni — Informatica Triennale

Docente: Luca Sciullo

Dipartimento di Informatica – Scienza e Ingegneria

A.A. 2024/2025

Contents

1	Introduzione al Linguaggio C	3
2	Gerarchia dei Linguaggi di Programmazione	3
3	Storia del Linguaggio C	3
3.1	Dai precursori al C	3
3.2	Evoluzione degli standard	3
4	Struttura di un Sistema Operativo	4
4.1	Kernel vs User Space	4
4.2	System Call	4
5	UNIX	4
5.1	Origini	4
5.2	Unix Philosophy	4
5.3	Discendenti	5
6	POSIX	5
7	Linux e GNU	5
7.1	Il kernel Linux	5
7.2	Progetto GNU	5
7.3	Distribuzioni principali	6
7.4	Linux nel mondo (2025)	6
8	Linguaggio Macchina e Assembler	6
8.1	Linguaggio macchina	6
8.2	Linguaggio assembler	6
8.3	Lo stesso algoritmo in C	6

9	Perche' C per i Sistemi Operativi	7
10	Tipi di Dato in C	7
10.1	Tipi primitivi	7
10.2	Problema ABI: LP64 vs LLP64	7
10.3	Soluzione: tipi a larghezza fissa	7
10.4	Specificatori printf	8
11	Puntatori e Gestione della Memoria	8
11.1	Operazioni fondamentali	8
11.2	Passaggio per valore vs per riferimento	8
11.3	Allocazione dinamica	9
11.4	Layout memoria di un processo	9
12	Struct e Union	10
12.1	Struct	10
12.1.1	Padding e allineamento	10
12.2	Union	10
13	Esercizi Proposti	11
13.1	Tipi di dato	11
13.2	Puntatori	11
13.3	Struct e Union	11
14	Riferimenti	11

Introduzione al Linguaggio C

Il **linguaggio C** è un linguaggio di programmazione **imperativo**, **strutturato** e **compilato**, nato negli anni '70 per lo sviluppo di sistemi operativi (in particolare UNIX).

Caratteristiche fondamentali:

- **Controllo esplicito della memoria:** il programmatore gestisce direttamente allocazione e deallocazione.
- **Portabilità:** lo stesso sorgente compila su architetture diverse.
- **Paradigma procedurale:** la logica è organizzata in *funzioni*, non in classi.
- **Efficienza:** nessun garbage collector nè overhead di runtime.

Ambiti applicativi principali: kernel di SO, driver, applicazioni embedded, librerie critiche (glibc, OpenSSL, stack di rete, database).

Gerarchia dei Linguaggi di Programmazione

Livello	Categoria
1	Hardware
2	Microlinguaggi
3	Linguaggio macchina
4	Linguaggio assembler
5	Linguaggi ad alto livello (C, C++, Java...)

Posizione del C

Il C si colloca al confine tra assembler e linguaggi ad alto livello: offre astrazioni (funzioni, tipi, strutture) mantenendo accesso diretto all'hardware tramite puntatori.

Storia del Linguaggio C

Dai precursori al C

$Algol \longrightarrow BCPL(Richards, 1966) \longrightarrow B(Thompson, 1969) \longrightarrow C(Ritchie, 1972)$

- **BCPL** (1966): linguaggio minimale per scrivere compilatori.
- **B** (1969, Ken Thompson): derivato da BCPL, privo di tipi.
- **C** (1972, Dennis Ritchie): aggiunge tipi, aritmetica dei puntatori, struct. Progettato per scrivere sistemi operativi.

Evoluzione degli standard

Anno	Evento
1972	Ritchie sviluppa C ai Bell Labs (AT&T).
1977	Kernighan & Ritchie: “The C Programming Language” (K&R C).
1989/90	C89/C90 : primo standard ANSI/ISO ufficiale.
1999	C99 : commenti <code>//</code> , <code>long long</code> , VLA, <code><stdint.h></code> .
2011	C11 : <code>_Atomic</code> , modello di concorrenza minimo.
2017	C17 : correzioni.
2023	C23 : aggiornamenti moderni.

Struttura di un Sistema Operativo

Kernel vs User Space

- **Kernel**: nucleo residente. Gestisce CPU, memoria, file system, dispositivi. Caricato al boot, rimane sempre in memoria.
- **User Space**: shell, librerie, utility. Accedono al kernel tramite **system call**.

System Call

Circa **400** system call in Linux, organizzate per categoria: file/dispositivi, rete, processi (`fork/exec/wait`), IPC, memoria, timer, thread, sincronizzazione.

Nota

Le applicazioni non invocano le system call direttamente: usano le funzioni della **libreria C** (`glibc`), che funge da interfaccia portabile.

UNIX

Origini

UNIX nasce ai **Bell Labs** (AT&T) tra fine anni '60 e inizio '70 (Ken Thompson, Dennis Ritchie). Caratteristiche:

- Multiutente, multitasking, portabile.
- “*Everything is a file*”.
- Design modulare e minimale. Scritto in C.

Unix Philosophy

Doug McIlroy (inventore delle Unix Pipe):

Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.

Discendenti

BSD (FreeBSD, OpenBSD, NetBSD), macOS/iOS (BSD + kernel XNU), AIX, HP-UX, Solaris.

Attenzione

“Unix” è un marchio registrato (The Open Group). Solo sistemi certificati (macOS, AIX) sono legalmente “Unix”. Linux è *Unix-like*.

POSIX

POSIX (*Portable Operating System Interface with X for Unix*) è una famiglia di standard IEEE (1003). Nome coniato da Richard Stallman.

Cosa definisce:

1. API: `fork`, `exec`, `open`, `read`, `write`, segnali, semafori, IPC.
2. Utility: `ls`, `cp`, `grep`, `awk`...
3. Shell Bourne (`sh`).

Listing 1: Codice POSIX portabile

```
1 int fd = open("file.txt", O_RDONLY);
2 char buf[100];
3 read(fd, buf, 100);
4 close(fd);
5 // Identico su Linux, macOS, FreeBSD
```

Concetto chiave

Linux implementa gran parte degli standard POSIX: è Unix-like ma non Unix.

Linux e GNU

Il kernel Linux

Kernel libero (GPL) creato da **Linus Torvalds** nel 1991.

Linux non è un SO completo da solo

Gestisce processi, memoria, driver, filesystem, rete. Servono anche: shell, compilatori, librerie, utility.

Oggi: **34 milioni** di righe di codice, oltre **10.000** contributori (Microsoft, Intel, IBM...).

Progetto GNU

Fondato da **Richard Stallman** (FSF, 1983). Produce: `glibc`, `GCC`, `Bash`, `Coreutils`, `Emacs`. Il kernel GNU Hurd non ebbe successo; si adottò Linux:

$$Linux + GNU = \mathbf{GNU/Linux}$$

Distribuzioni principali

Distro	Target	Pro	Contro	Pkg
Ubuntu	Desktop/Cloud	Facile, LTS	SW non fresco	apt
Debian	Server	Rock-solid	Conservative	apt
Fedora	Sviluppatori	Cutting-edge	Poco stabile	dnf
RHEL	Enterprise	Supporto comm.	A pagamento	dnf
Arch	Power users	Rolling, KISS	Curva ripida	pacman

Linux nel mondo (2025)

49,2% workload cloud globali; 100% dei top-500 supercomputer; 78,5% degli sviluppatori lo usa; 3,7% del mercato desktop.

Linguaggio Macchina e Assembler

Linguaggio macchina

Sequenze di bit eseguite direttamente dalla CPU. Ogni istruzione: **opcode** (operazione) + **operando** (dato/indirizzo).

00000010 0000011011100 ; carica cella 220 in accumulatore

00000110 0000011111100 ; somma cella 252

00000100 0000011011100 ; memorizza in cella 220

Non portabile

Gli opcode variano da CPU a CPU. Il codice macchina e' strettamente legato all'architettura hardware.

Linguaggio assembler

Mnemonici leggibili tradotti automaticamente in codice oggetto:

LOAD 220

SUM 252

MEM 220

Codice mnemonico assembler → Codice oggetto → Esecuzione

Lo stesso algoritmo in C

Listing 2: Moltiplicazione come somme ripetute

```
1 #include <stdio.h>
2 int main() {
3     int a = 6, b = 4, c = 0;
4     for (int i = 0; i < b; i++)
5         c = c + a;
6     printf("%d\n", c); // 24
7     return 0;
```

Perche' C per i Sistemi Operativi

Concetto chiave

C nasce *per* Unix; Unix si diffonde *grazie* a C. Le system call POSIX sono definite tramite interfacce C. I manuali **man 2** usano prototipi C come specifica de facto.

Motivi tecnici:

1. **Portabilita'**: stesso sorgente su architetture diverse.
2. **Gestione memoria esplicita**: nessun GC.
3. **Puntatori**: accesso diretto all'hardware.
4. **ABI stabile**: librerie interoperabili tra linguaggi.
5. **Prestazioni prevedibili**: nessun JIT, nessuna GC pause.

Tipi di Dato in C

Tipi primitivi

Lo standard fissa solo l'ordinamento: $\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long})$

Tipo	32-bit	Linux 64	Win 64
char	1B	1B	1B
short	2B	2B	2B
int	4B	4B	4B
long	4B	8B	4B
long long	8B	8B	8B
pointer	4B	8B	8B

Problema ABI: LP64 vs LLP64

Attenzione

LP64 (Linux/Unix 64-bit): `long` e puntatori = 64 bit.

LLP64 (Windows 64-bit): `long` = 32 bit, puntatori = 64 bit.

Conseguenze: strutture di dimensione diversa, overflow inattesi, `printf` non portabile.

Soluzione: tipi a larghezza fissa

Listing 3: `stdint.h` e `inttypes.h`

```

1 #include <stdint.h>
2 #include <inttypes.h>
3 #include <stdio.h>
```

```

4
5 int main() {
6     int32_t a = 100000; Sarà sempre un intero con segno da esattamente 32 bit (4 byte)
7     uint64_t b = 180000000000ULL; uint64_t: Sarà sempre un intero senza segno (la 'u' sta per unsigned) da esattamente 64 bit (8 byte) ovunque. ?
8     size_t n = sizeof(b);
9
10    printf("a = %" PRId32 "\n", a);
11    printf("b = %" PRId64 "\n", b);
12    printf("n = %zu\n", n);
13    return 0;
14 }

```

Come stamparli a schermo (Specificatori printf)

Visto che i tipi sono "fissi" indipendentemente dal sistema, anche il modo in cui diciamo alla funzione `printf` di stamparli deve adattarsi. Non possiamo più usare il classico `%d` o `%ld`.

La libreria `<inttypes.h>` fornisce delle macro apposite (che sono fondamentalmente delle costanti di testo) per la `printf`: [?](#)

- Per stampare un `int32_t`, devi usare la macro `PRId32`. Si scrive così:
`printf("Valore = %" PRId32 "\n", variabile);` [?](#)
- Per stampare un `uint64_t`, devi usare la macro `PRId64`. [?](#)
- Per stampare un `size_t`, lo standard `<stdio.h>` fornisce lo specificatore apposito `%zu`.

Tipi speciali importanti: `size_t` (dimensioni), `ptrdiff_t` (differenze tra puntatori), `off_t` (offset su file).

Specificatori printf

Tipo	Header	Specificatore
int	stdio.h	%d / %u
long	stdio.h	%ld / %lu
long long	stdio.h	%lld / %llu
size_t	stdio.h	%zu
int32_t	inttypes.h	%" PRId32 "
uint64_t	inttypes.h	%" PRId64 "
void*	stdio.h	%p

Puntatori e Gestione della Memoria

Operazioni fondamentali

Listing 4: Puntatori – basi

```

1 int x = 42;
2 int *p = &x;           // p = indirizzo di x
3
4 printf("%d\n", x);      // 42 -- valore variabile
5 printf("%p\n", &x);     // indirizzo di x
6 printf("%p\n", p);      // uguale a &x
7 printf("%d\n", *p);     // 42 -- dereferenziazione
8
9 *p = 100;               // modifica x tramite puntatore
10 printf("%d\n", x);     // 100

```

Passaggio per valore vs per riferimento

In C le funzioni ricevono argomenti **per valore**. Per modificare la variabile originale si passa un puntatore:

Listing 5: swap corretto


```

1 void swap(int *x, int *y) {
2     int tmp = *x;  *x = *y;  *y = tmp;
3 }
4 int main() {
5     int a = 5, b = 10;
6     swap(&a, &b);
7     printf("a=%d b=%d\n", a, b);  // a=10 b=5
8 }

```

Allocazione dinamica

Listing 6: malloc, calloc, free

```

1 #include <stdlib.h>
2 #include <string.h>
3
4 int *genera_array(int n) {
5     int *arr = (int *)malloc(n * sizeof(int));
6     if (!arr) return NULL;
7     memset(arr, 0, n * sizeof(int));
8     return arr;
9 }
10
11 // Matrice 2D
12 int **crea_matrice(int r, int c) {
13     int **mat = (int **)malloc(r * sizeof(int *));
14     for (int i = 0; i < r; i++)
15         mat[i] = (int *)malloc(c * sizeof(int));
16     return mat;
17 }
18 void libera_matrice(int **mat, int r) {
19     for (int i = 0; i < r; i++) free(mat[i]);
20     free(mat);
21 }

```

Errori comuni

Memory leak: dimenticare `free()`.
Dangling pointer: usare puntatore dopo `free()`.
Buffer overflow: scrivere oltre i limiti.
Double free: liberare due volte la stessa area.

Layout memoria di un processo

Segmento	Contenuto
Stack (↓, indirizzi alti)	Variabili locali, frame di funzione
Heap (↑)	Memoria allocata con malloc
BSS	Var. globali non inizializzate
Data	Var. globali inizializzate
Text (indirizzi bassi)	Codice eseguibile

Regole per i padding

Perché serve il Padding?

La CPU non legge la memoria un byte alla volta. Legge la memoria a "blocchi" (word), solitamente di 4 o 8 byte alla volta. Se una variabile (ad esempio un `int` di 4 byte) inizia a metà di uno di questi blocchi e finisce nel blocco successivo, la CPU dovrà fare **due** letture della memoria per recuperare un solo numero, rallentando il programma.

Per evitare questo, il compilatore aggiunge dei byte "vuoti" o "spazzatura" (il padding) tra una variabile e l'altra all'interno della `struct` per far sì che le variabili più grandi "cadano" sempre all'inizio di un blocco di memoria ottimale.

Le due regole d'oro dell'Allineamento

Per capire i calcoli dell'immagine, devi tenere a mente due regole:

1. **Allineamento dei singoli tipi:** Ogni tipo primitivo deve iniziare in un indirizzo di memoria che sia multiplo della sua dimensione.
 - `char` (1 byte): può iniziare ovunque (multiplo di 1).
 - `short` (2 byte): deve iniziare in un indirizzo multiplo di 2 (es. offset 0, 2, 4, 6...).
 - `int` (4 byte): deve iniziare in un indirizzo multiplo di 4 (es. offset 0, 4, 8, 12...).
2. **Allineamento totale della Struct:** La dimensione *totale* della `struct` deve essere un multiplo del requisito di allineamento più grande tra i suoi membri. Se il membro più grande è un `int` (4 byte), la struct totale dovrà avere una dimensione multipla di 4.

Qui a seguire vediamo l'analisi dei padding aggiunti alle strutture/union dei codici degli esempi riportati subito a seguito delle analisi

Analizziamo i casi dell'immagine passo passo

Caso 1: struct Data1 (L'ordine peggiore)

c



```
struct Data1 { char a; int b; short c; };
```

- `char a` (1 byte): Viene messo all'inizio (offset 0). Occupa 1 byte.
- `int b` (4 byte): La regola dice che un `int` deve iniziare a un multiplo di 4. Attualmente siamo all'offset 1. Il prossimo multiplo di 4 è 4. Quindi il compilatore inserisce **3 byte di padding** (offset 1, 2, 3) per far iniziare `b` all'offset 4. Ora siamo a 8 byte occupati totali.
- `short c` (2 byte): Deve iniziare a un multiplo di 2. Attualmente siamo all'offset 8. L'8 è già un multiplo di 2, quindi lo inseriamo direttamente. Nessun padding intermedio. Ora siamo a 10 byte occupati totali.
- **Regola della Struct totale:** Il tipo più grande all'interno è l'`int` (4 byte). La dimensione totale (che ora è 10) deve essere un multiplo di 4. Il prossimo multiplo di 4 dopo il 10 è il 12. Quindi il compilatore aggiunge altri **2 byte di padding finali**.
- **Totale:** 1 (`char`) + 3 (`pad`) + 4 (`int`) + 2 (`short`) + 2 (`pad finale`) = 12 byte.

Caso 2: struct Data2 (L'ordine ottimizzato)

Se ordini i campi dal più grande al più piccolo, risparmi molta memoria.

c



```
struct Data2 { int b; short c; char a; };
```

- `int b` (4 byte): Messo all'inizio (offset 0).
- `short c` (2 byte): Siamo all'offset 4. Il 4 è multiplo di 2? Sì. Nessun padding. Inseriamo `c`. Ora siamo a 6 byte occupati.
- `char a` (1 byte): Siamo all'offset 6. Il `char` va bene ovunque. Inseriamo `a`. Ora siamo a 7 byte occupati.
- **Regola della Struct totale:** Il tipo più grande è l'`int` (4 byte). Il totale è 7. Per arrivare al prossimo multiplo di 4 (che è 8), il compilatore aggiunge **1 byte di padding finale**.
- **Totale:** 4 (`int`) + 2 (`short`) + 1 (`char`) + 1 (`pad finale`) = 8 byte.

Caso 3: #pragma pack(1)

c



```
#pragma pack(1)
struct DataP { char a; int b; short c; };
#pragma pack()
```

Questa istruzione speciale (`#pragma pack(1)`) dice letteralmente al compilatore: *"Ignora tutte le regole di allineamento della CPU e compatta tutto al massimo usando un allineamento di 1 byte, non mi interessano le prestazioni".*

Senza le regole di allineamento, non viene inserito **nessun padding**.

- **Totale:** 1 (`char`) + 4 (`int`) + 2 (`short`) = 7 byte.

In sintesi

Il padding non è un errore, ma un'ottimizzazione silenziosa che fa il compilatore.

L'insegnamento principale di queste slide è: **quando crei una struct in C, metti sempre prima le variabili che occupano più byte (come `double`, puntatori e `int`) e alla fine quelle più piccole (`char`, `bool`).** In questo modo minimizzerai i "buchi" vuoti nella memoria!

Struct e Union

Struct

Listing 7: Struct annidate e calcolo area

```
1 typedef struct { float x; float y; } Point;
2
3 typedef struct {
4     Point top_left;
5     Point bottom_right;
6 } Rectangle;
7
8 float area(Rectangle r) {
9     float base      = r.bottom_right.x - r.top_left.x;
10    float altezza    = r.bottom_right.y - r.top_left.y;
11    return base * altezza;
12 }
```

12.1.1 Padding e allineamento

Il compilatore aggiunge *padding* per rispettare l'allineamento della CPU. L'ordine dei campi influenza la dimensione totale:

Listing 8: Effetto dell'ordine dei campi

```
1 struct Data1 { char a; int b; short c; };
2 // Layout: char(1)+pad(3)+int(4)+short(2)+pad(2) = 12 byte
3
4 struct Data2 { int b; short c; char a; };
5 // Layout: int(4)+short(2)+char(1)+pad(1) = 8 byte
6
7 #pragma pack(1)
8 struct DataP { char a; int b; short c; }; // 7 byte (no padding)
9 #pragma pack()
```

Union

Condivide la stessa area di memoria tra tutti i campi:

Listing 9: Union per ispezione dei byte interni

```
1 typedef union {
2     int      i;
3     float    f;
4     unsigned char bytes[4];
5 } Number;
6
7 void stampa_bytes(Number n) {
8     for (int j = 0; j < 4; j++) printf("%02X ", n.bytes[j]);
9     printf("\n");
10 }
11 int main() {
```

```

12     Number n;
13     n.i = 12345;    printf("int:   "); stampa_bytes(n);
14     n.f = 3.14f;    printf("float: "); stampa_bytes(n);
15 }

```

Esercizi Proposti

Tipi di dato

1. Stampa dimensione e range di `char`, `int`, `float`, `double` con `<limits.h>` e `<float.h>`.
2. Dichiarare `intmax_t` e stampala con `PRIdMAX`.
3. Analizza `unsigned u=0`; `u-1` e il confronto `(int)-1 < u`.
4. Verifica a runtime se `char` e' signed o unsigned.
5. Implementa permessi con `enum` e bitmask (`READ=1`, `WRITE=2`, `EXEC=4`).

Puntatori

1. Stampa valore, indirizzo e valore puntato di un intero.
2. Implementa `swap(int *x, int *y)`.
3. Implementa `int* genera_array(int n)`.
4. Alloca, riempi, stampa e libera una matrice $n \times m$.
5. Lista concatenata: `insert_head`, `print_list`, `free_list`.

Struct e Union

1. `Rectangle`: calcola base, altezza e area.
2. Confronta `sizeof` di due struct con campi in ordine diverso; applica `#pragma pack`.
3. `Union Number`: ispezione dei byte interni per `int` e `float`.

Riferimenti

- B.W. Kernighan, D.M. Ritchie — *The C Programming Language*, 2nd ed., Prentice Hall, 1988.
- S.P. Harbison, G.L. Steele — *C: A Reference Manual*, 5th ed.
- E.S. Raymond — *The Art of UNIX Programming*, Addison-Wesley, 2003.
<https://cdn.nakamotoinstitute.org/docs/taoup.pdf>
- A. Bellini, A. Guidi — *Linguaggio C: guida alla programmazione*, 4a ed. McGraw-Hill.
- Princeton COS 217 – Data Types:
https://www.cs.princeton.edu/courses/archive/spr18/cos217/lectures/04_DataTypes.pdf

- Puntatori (Polito):
<https://staff.polito.it/claudio.fornaro/CorsoC/13-Puntatori.pdf>
- Kernel Linux: <https://github.com/torvalds/linux>
- Distrowatch: <https://distrowatch.com>

Note finali sul perchè servano i bit di padding aggiunti dalla CPU

 a prima vista sembra solo una complicazione e un inutile spreco di RAM!

Ma c'è un motivo puramente "fisico" e hardware legato a **come la CPU comunica con la memoria**. Il compilatore aggiunge questo padding proprio per rispettare l'allineamento richiesto dalla CPU. 

La CPU è "ottimizzata" (o pigra)

Noi pensiamo alla memoria come a una lunghissima fila di byte singoli, e crediamo che la CPU possa prendere un byte qualsiasi in un millisecondo. Nella realtà, la CPU non legge la memoria un byte alla volta.

Per essere ultra-veloce, la CPU legge e scrive in RAM a "blocchi" di dimensioni fisse (solitamente di 4 o 8 byte alla volta), chiamati *Word*.

L'analogia della scatola e del libro

Immagina che la memoria sia un libro, e ogni pagina può contenere esattamente 4 lettere. Un numero intero (`int`) è una parola di 4 lettere.

- **Con il padding (Allineato):** La parola inizia esattamente all'inizio di una pagina vuota. Tu apri il libro, guardi la pagina, e leggi la parola in **un colpo d'occhio**.
- **Senza padding (Disallineato):** La parola inizia sull'ultima lettera di pagina 1, e continua con le altre tre lettere su pagina 2. Per leggerla, devi guardare pagina 1, ricordarti il primo pezzo, **girare pagina**, guardare pagina 2, prendere il secondo pezzo e "incollarli" nella tua mente.

Cosa succede a livello hardware se NON c'è padding?

Se un dato da 4 byte (come un `int`) finisce a cavallo tra due blocchi di memoria perché non hai messo il padding prima di lui:

1. **Danno alle prestazioni (Rallentamento):** La CPU è costretta a fare due letture dalla RAM invece di una. Poi deve fare operazioni extra per "tagliare e incollare" i bit giusti. Questo dimezza la velocità di accesso a quella variabile.
2. **Crash del sistema:** Su alcune architetture hardware (come molti processori ARM, usati negli smartphone, o sistemi *embedded*), la CPU si rifiuta categoricamente di fare questa doppia lettura. Se provi a leggere un `int` non allineato, l'hardware va in panico e fa crashare il programma immediatamente (generando un errore chiamato *Bus Error*). 

Il compromesso del C: Spazio vs Velocità

Il linguaggio C è nato negli anni '70 specificamente per scrivere Sistemi Operativi, programmi che devono spremere ogni singola goccia di performance dall'hardware.

Il compilatore C fa una scelta drastica ma consapevole: **sacrifica un pochino di memoria** (aggiungendo byte vuoti di padding) per assicurarsi che i dati siano sempre perfettamente impaginati per la CPU. In questo modo, l'hardware non fa mai fatica e il tuo programma corre alla **massima velocità possibile**.